



Rapport de mini-projet C++ : gestionnaire de taches
(Todo List)

Réalisé par :

- Faragi Marwa
- Saaad Romaissa
- Essalihi Hajar

Encadré par :

- M. Youssef ES-SAADY
- Mlle. AICHA DAKIR

Année universitaire :

- 2025 – 2026

Introduction :

Dans le cadre du module de programmation orientée objet en `c++`, ce mini projet a été réalisé afin de mettre en pratique les différentes notions théoriques étudiées durant les séances de cours, de Tds et de Tps.

L'objectif principal de ce travail est de concevoir un système de gestion de tâches fonctionnant en mode console, permettant à l'utilisateur d'organiser effectivement ses tâches personnelles à travers plusieurs fonctionnalités telles que l'ajout, la modification, la suppression, le tri et la sauvegarde des données.

La réalisation de ce projet nous a permis d'approfondir nos connaissances en programmation orientée objet, à travers l'utilisateur des classes et objets, l'encapsulation des données, l'héritage, le polymorphisme ainsi que la gestion des fichiers en `c++`.

Par ailleurs, ce travail nous a permis de développer des compétences en organisation et structuration d'un projet informatique, en adoptant une architecture claire et modulaire facilitant la maintenance et l'évolution de l'application.

Présentation du projet :

Le projet est réalisé consiste en un gestionnaire de tâches développé en **c++** et fonctionnant en mode console. Cette application a été conçue dans le but de faciliter l'organisation et le suivi des tâches personnelles et professionnelles.

Le système permet à l'utilisateur d'ajouter, modifier, supprimer, archiver et consulter différentes tâches selon plusieurs critères tels que la priorité, la date limite ou le statut d'avancement.

Le projet intègre également plusieurs fonctionnalités avancées, notamment la gestion des tâches récurrentes, un système de notifications pour les tâches urgentes ou en retard, des statistiques générales ainsi qu'un support multilingues (**français et anglais**).

Afin d'assurer une bonne organisation du projet, l'architecture a été séparée en plusieurs fichiers **d'en-tête (.h)** et fichiers d'implémentation (**.cpp**), facilitant ainsi la maintenance, la lisibilité et l'évolution du code.

Principales fonctionnalités :

- ⇒ Gestion des tâches personnelles et professionnelles
- ⇒ Gestion des tâches récurrentes
- ⇒ Définition des priorités et des dates limites
- ⇒ Archivage des tâches
- ⇒ Tri et filtrage des tâches
- ⇒ Notification des tâches urgentes
- ⇒ Sauvegarde des données dans des fichiers
- ⇒ Support multilingue
- ⇒ Génération de statistiques
- ⇒ Suivi de l'humeur
- ⇒ Affichage de citations et messages de motivation

Structure du projet :

Le projet a été conçu selon une architecture molaire afin d'assurer une bonne organisation du code et une séparation claire entre les différentes composantes de l'application. Cette approche permet de rendre le projet plus lisible, plus maintenable et plus facile à faire évoluer au fil du développement.

L'architecture du projet est divisée en plusieurs dossiers principaux, chacun ayant un rôle spécifique dans le fonctionnement global du système.

Le dossier ***include/*** regroupe l'ensemble des fichiers ***d'en-tête (.h)***. Ces fichiers contiennent principalement les déclarations des classes, des attributs et des méthodes utilisées dans l'application.

Ils définissent également les interfaces nécessaires à la communication entre les différentes composantes du projet. Cette séparation permet de mieux structurer le code et d'éviter les dépendances inutiles.

Le dossier ***src/*** contient les fichiers source (***.cpp***) responsables de l'implémentation de la logique du programme. Chaque classe possède généralement son propre fichier d'implémentation afin de faciliter la compréhension, la naissance et la réutilisation du code.

Cette organisation améliore également le travail collaboratif entre les membres du groupe en permettant à chacun de travailler sur des parties différentes du projet.

Le dossier ***data/*** est utilisé pour le stockage des données de l'application. Ils contiennent notamment des fichiers au format **JSON** permettant la sauvegarde des tâches, des paramètres ainsi que des fichiers de traduction utilisés pour le support multilingue. Grâce à ce système, les données restent conservées même après la fermeture du programme.

Le projet utilise également **CMake** comme système de compilation sur différents environnements de développement.

Cette solution facilite la probabilité du projet et garantit une compilation plus propre et mieux organisée.

Enfin, l'utilisateur de **GitHub** a permis de gérer efficacement le développement du projet grâce au suivi des versions, à l'historique des modifications et au travail collaboratif entre les différents membres de l'équipe.

Description des classes et architecture objet :

La conception du projet repose sur plusieurs classes permettant d'organiser les différentes fonctionnalités du gestionnaire de tâches. Chaque classe possède un rôle bien défini afin de garantir une architecture claire, modulaire et facile à maintenir.

La classe principale **Task** représente une tâche individuelle dans l'application. Elle contient les informations essentielles liées à une tâche, telles que le titre, la description, la priorité, le statut ainsi que la date limite. Cette classe regroupe également plusieurs méthodes permettant d'afficher, modifier et gérer les différentes propriétés d'une tâche.

Afin d'améliorer l'organisation du système, plusieurs types de tâches spécialisées ont été implémentés, notamment **WorkTask**, **PersonalTask** et **RecurringTask**. Ces classes héritent de la classe de base **Task** et permettent d'ajouter des comportements spécifiques selon le type de tâche traité. Par exemple, la classe **RecurringTask** permet de gérer les tâches répétitives grâce à un système de récurrence quotidienne, hebdomadaire ou mensuelle. Cette approche facilite la réutilisation du code et illustre l'utilisation du principe d'héritage en programmation orientée objet.

La classe **Deadline** est utilisée pour gérer les dates limites des tâches. Elle permet de stocker le jour, le mois et l'année afin de faciliter la gestion des échéances et la détection des tâches en retard.

La classe **TaskManager** joue un rôle central dans l'application. Elle est responsable de la gestion complète des tâches, notamment l'ajout, la suppression, la restauration, l'archivage, l'affichage ainsi que la modification des tâches enregistrées. Cette classe centralise toutes les opérations principales du système.

Le projet intègre également une classe **Archive** chargée de gérer les tâches archivées. Elle permet de stocker, afficher et supprimer définitivement les tâches archivées.

La classe ***FileManager*** assure la sauvegarde et le chargement des données à partir des fichiers de stockage. Elle garantit la conservation des tâches et des archives même après la fermeture de l'application.

Plusieurs classes utilitaires ont également été développées afin d'améliorer les fonctionnalités du système. La classe **TaskFilter** permet de filtrer les tâches selon différents critères comme le statut, la priorité ou les tâches en retard. La classe **TaskSorter** permet de trier les tâches par priorité ou par date limite. La classe **TaskStats** est utilisée pour calculer et afficher les statistiques liées aux tâches et à la productivité.

Le projet possède aussi une classe **NotificationManager** chargée de gérer les notifications des tâches importantes, urgentes ou proches de leur deadline. La classe **TodayView** permet d'afficher uniquement les tâches prévues pour la journée actuelle afin de faciliter l'organisation quotidienne.

Afin d'améliorer l'expérience utilisateur, le système intègre une classe **LanguageManager** permettant de gérer plusieurs langues à travers des fichiers JSON de traduction. De plus, les classes **MoodTracker** et **MotivationManager** permettent respectivement de suivre l'humeur quotidienne de l'utilisateur et d'afficher des citations motivantes adaptées à son état émotionnel.

L'ensemble de ces classes collabore afin d'assurer le bon fonctionnement du système tout en respectant les principes fondamentaux de la programmation orientée objet tels que l'encapsulation, l'héritage, le polymorphisme, la modularité et la réutilisation du code.

Fonctionnement général du système :

Le fonctionnement général de l'application repose sur un système de gestion de tâches en mode console permettant à l'utilisateur d'interagir facilement avec les différentes fonctionnalités proposées.

Lors du démarrage de l'application, l'utilisateur est invité à sélectionner la langue d'utilisation du système (français ou anglais). Une fois la langue choisie, le menu principal s'affiche, offrant un accès aux différentes fonctionnalités disponibles. À partir de ce menu, l'utilisateur peut ajouter de nouvelles tâches, modifier ou supprimer des tâches existantes, ainsi que consulter la liste complète des tâches enregistrées.

Chaque tâche contient plusieurs informations importantes telles que le titre, la description, la catégorie, la priorité et le statut. Ces informations permettent une meilleure organisation des tâches et facilitent leur suivi.

Le système intègre également des fonctionnalités de tri et de filtrage afin d'améliorer la gestion des données et de permettre un accès plus rapide aux tâches importantes ou urgentes.

Les données de l'application sont sauvegardées dans des fichiers **JSON** afin d'assurer la conservation des informations même après la fermeture du programme. Lors du redémarrage de l'application, les données sauvegardées sont automatiquement rechargées.

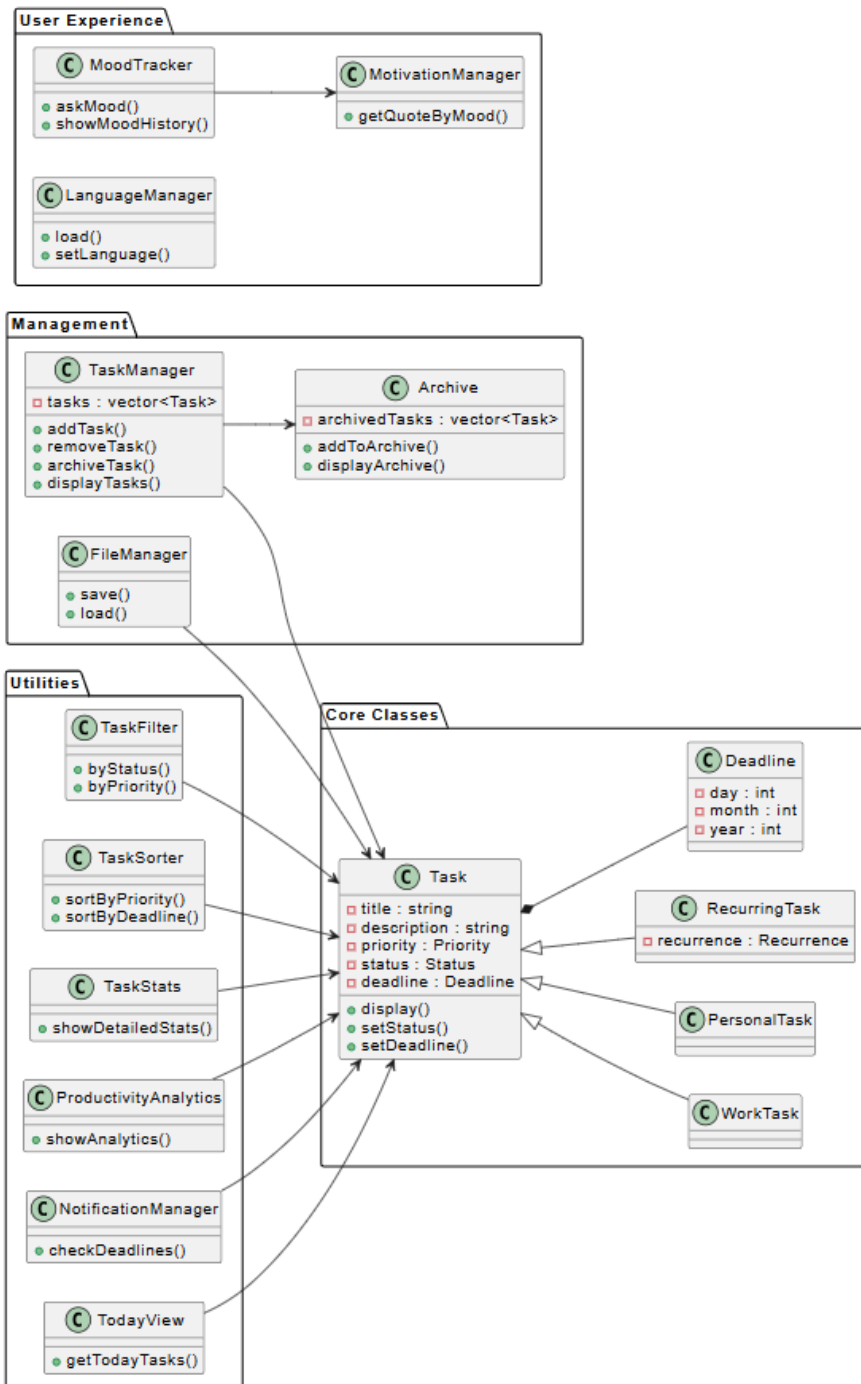
Grâce à son architecture modulaire et à l'utilisation des concepts de programmation orientée objet, le système reste organisé, évolutif et facile à maintenir.

Concepts POO utilisés :

Dans ce projet, la programmation orientée objet (**POO**) a été utilisée pour organiser le code de manière claire, structurée et facile à maintenir.

- **Classes et objets** : le programme est construit autour de classes qui représentent les éléments principaux du système, comme les tâches et leur gestion. Chaque tâche est considérée comme un objet avec ses propres informations.
- **Encapsulation** : les données (comme le titre, la description, la priorité, etc.) sont protégées à l'intérieur des classes. Elles ne sont pas accessibles directement, mais uniquement via des méthodes, ce qui permet de sécuriser les informations et d'éviter les erreurs.
- **Méthodes** : les classes contiennent des méthodes qui permettent de manipuler les tâches, par exemple : ajouter une tâche, modifier ses informations, la supprimer ou l'afficher.
- **Modularité** : le code est organisé en plusieurs parties indépendantes, ce qui rend le projet plus propre et facilite la compréhension ainsi que la maintenance.
- **Réutilisation du code** : certaines fonctions et méthodes sont réutilisées dans différentes parties du programme afin d'éviter les répétitions et de rendre le code plus efficace.

Diagramme de classe :



Difficultés rencontrées :

Au cours du développement de ce projet, plusieurs difficultés techniques ont été rencontrées :

- Gestion de la mémoire dynamique :

L'une des principales difficultés concernait l'utilisation des pointeurs et de l'allocation dynamique de mémoire avec **new**.

Les tâches étant créées dynamiquement (**WorkTask**, **PersonalTask**, **RecurringTask**), il fallait gérer correctement leur suppression afin d'éviter les fuites de mémoire (**memory leaks**) ou les erreurs liées à la suppression d'objets.

- Gestion de la sauvegarde des données :

La mise en place du système de sauvegarde et de chargement des tâches à travers la classe **FileManager** a également représenté un défi.

Il fallait s'assurer que les tâches actives ainsi que les tâches archivées soient correctement enregistrées puis restaurées au redémarrage de l'application.

- Conception orientée objet :

L'organisation des différentes classes en utilisant les principes de la programmation orientée objet a demandé une réflexion importante.

Il fallait établir une structure claire entre la classe principale Task et ses classes dérivées (**WorkTask**, **PersonalTask**, **RecurringTask**) tout en évitant la duplication de code.

- Gestion des entrées utilisateur

L'utilisation de **cin** et **getline()** a causé plusieurs problèmes liés à la lecture des entrées utilisateur, notamment avec les retours à la ligne.

Pour résoudre ce problème, une fonction **clearInput()** a été mise en place afin de nettoyer correctement le buffer d'entrée.

- Implémentation de fonctionnalités avancées :

L'ajout de fonctionnalités supplémentaires comme :

- ✓ Le système d'archivage
- ✓ La restauration après suppression
- ✓ Les notifications de deadlines
- ✓ Le suivi d'humeur
- ✓ Les citations motivationnelles
- ✓ Le support multilingue

a augmenté la complexité globale du projet et a nécessité une bonne coordination entre plusieurs classes.

Conclusion :

Ces difficultés ont permis d'améliorer nos compétences en programmation orientée objet, en gestion de fichiers, ainsi qu'en organisation d'un projet C++ structuré et complet.

Remerciements :

Nous tenons à exprimer nos sincères remerciements à notre professeur M. Youssef ES-SAADY ainsi qu'à Mlle AICHA DAKIR pour leurs efforts, leurs conseils précieux et leur soutien. Grâce à leur engagement et à leurs explications, nous avons pu développer nos compétences et réaliser ce projet dans de bonnes conditions.